Plateforme de gestion des travaux dirigés

Rapport de modélisation UML et d'implémentation C++

Travaux pratiques — Génie logiciel

Université / Établissement : .....

Membres du groupe : .....

Année académique : 2026–2027

1. Contexte et problématique

Dans les universités béninoises, les responsables de TD doivent coordonner les étudiants, les groupes, les matières, les salles et les enseignants. La gestion manuelle rend les horaires et le suivi des présences difficiles à contrôler. La plateforme proposée centralise ces informations et aide à programmer les séances sans conflits.

2. Objectifs et périmètre fonctionnel

Le système doit gérer les matières, groupes et affectations d'étudiants; programmer les séances en choisissant salle et enseignant; consulter le calendrier; saisir les présences et produire un bilan.

Le prototype C++ fourni démontre ces opérations avec des données d'exemple. Il teste les chevauchements de salle, d'enseignant et de groupe et recherche une salle disponible pour un créneau donné. Les données ne sont pas conservées après la fermeture du programme.

3. Diagramme de cas d'utilisation

Le responsable gère les matières, groupes, affectations, séances et bilans. L'enseignant consulte le calendrier et saisit les présences. L'étudiant consulte le calendrier et son relevé. Programmer une séance inclut le choix des ressources et la vérification des disponibilités.

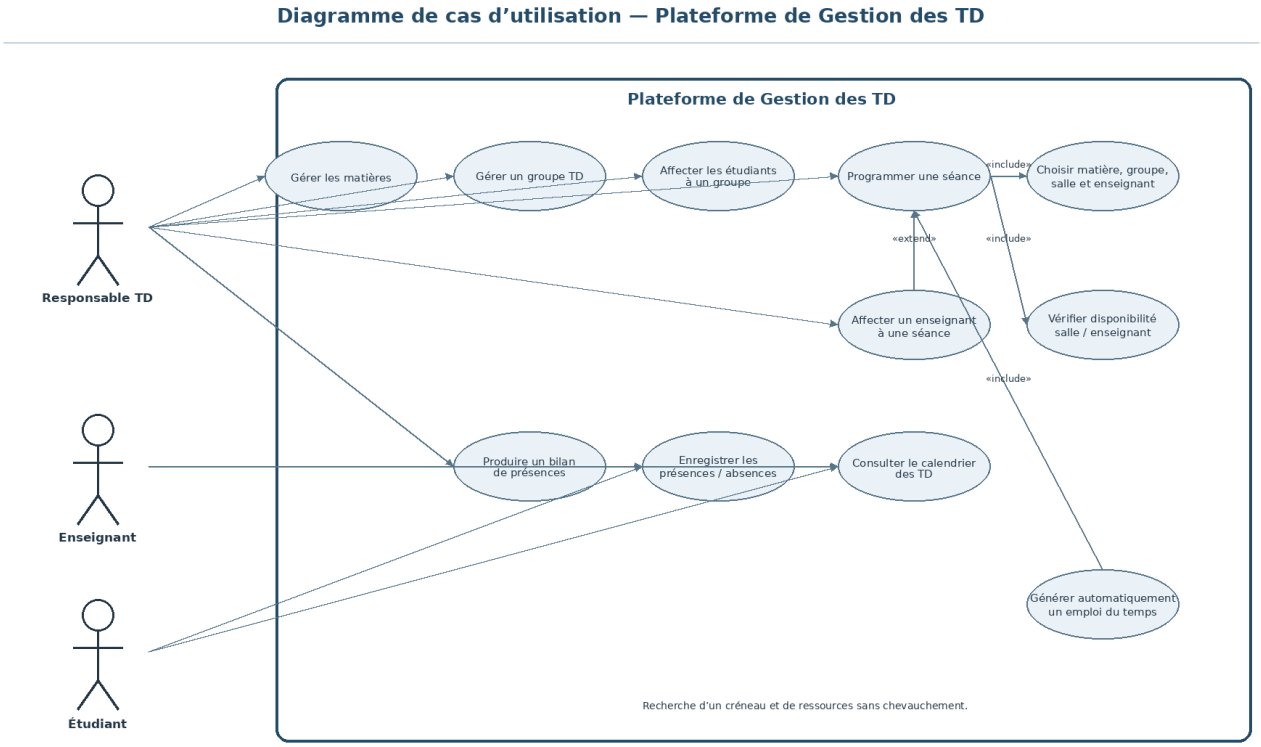


Diagramme 1 — Cas d'utilisation

4. Diagramme de classes

Les entités principales sont Etudiant, Matiere, GroupeTD, SeanceTD, Enseignant, Salle et Presence. Presence relie un étudiant à une séance et conserve le statut présent, absent ou retard. Chaque séance concerne un seul groupe, une matière, un enseignant et une salle; les contrôles empêchent les chevauchements.

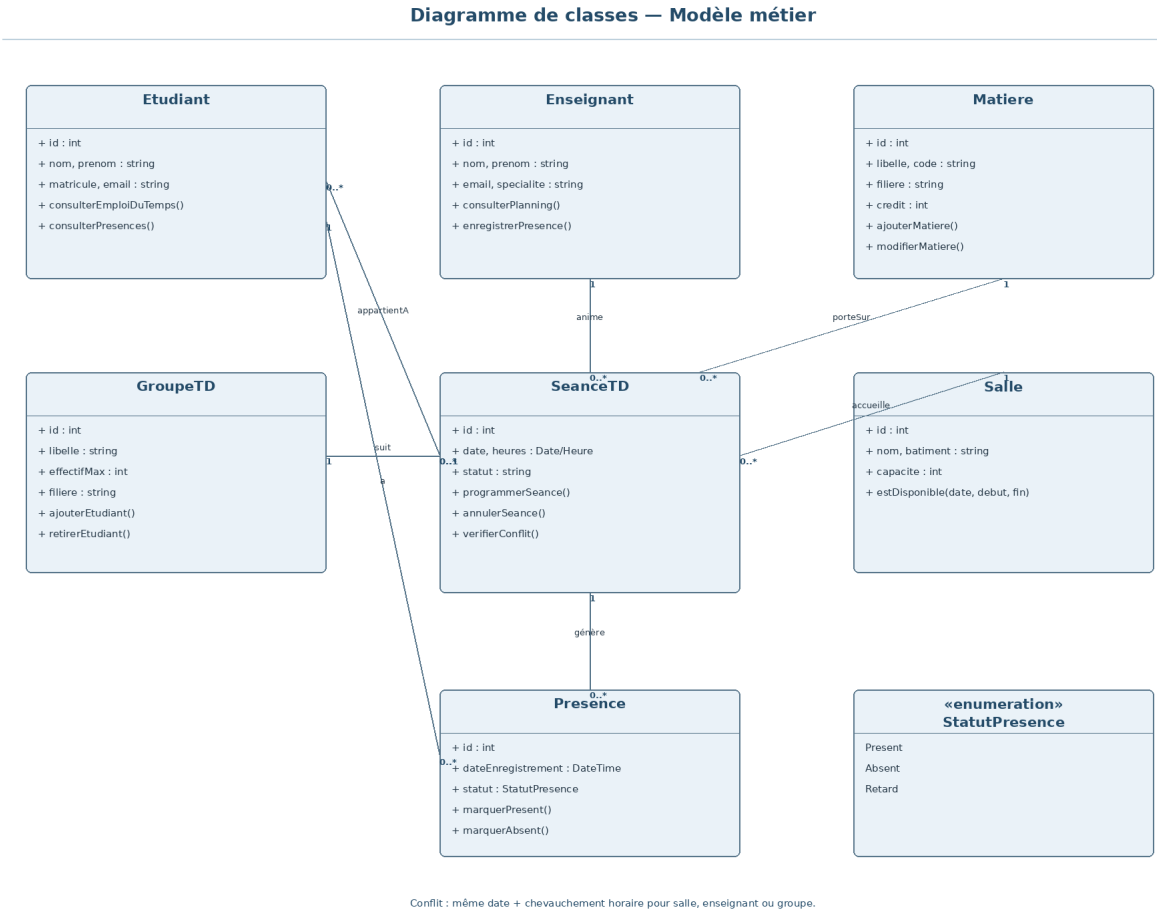


Diagramme 2 — Classes, attributs, opérations et associations

5. Diagramme de séquence : programmer une séance

Le responsable saisit les paramètres. Le contrôleur vérifie la salle et l'enseignant. Si les deux sont disponibles, il crée puis enregistre la séance et confirme l'opération. En cas de conflit, il retourne un message d'erreur. Dans le prototype console, la logique métier de PlateformeTD remplit le rôle du contrôleur.

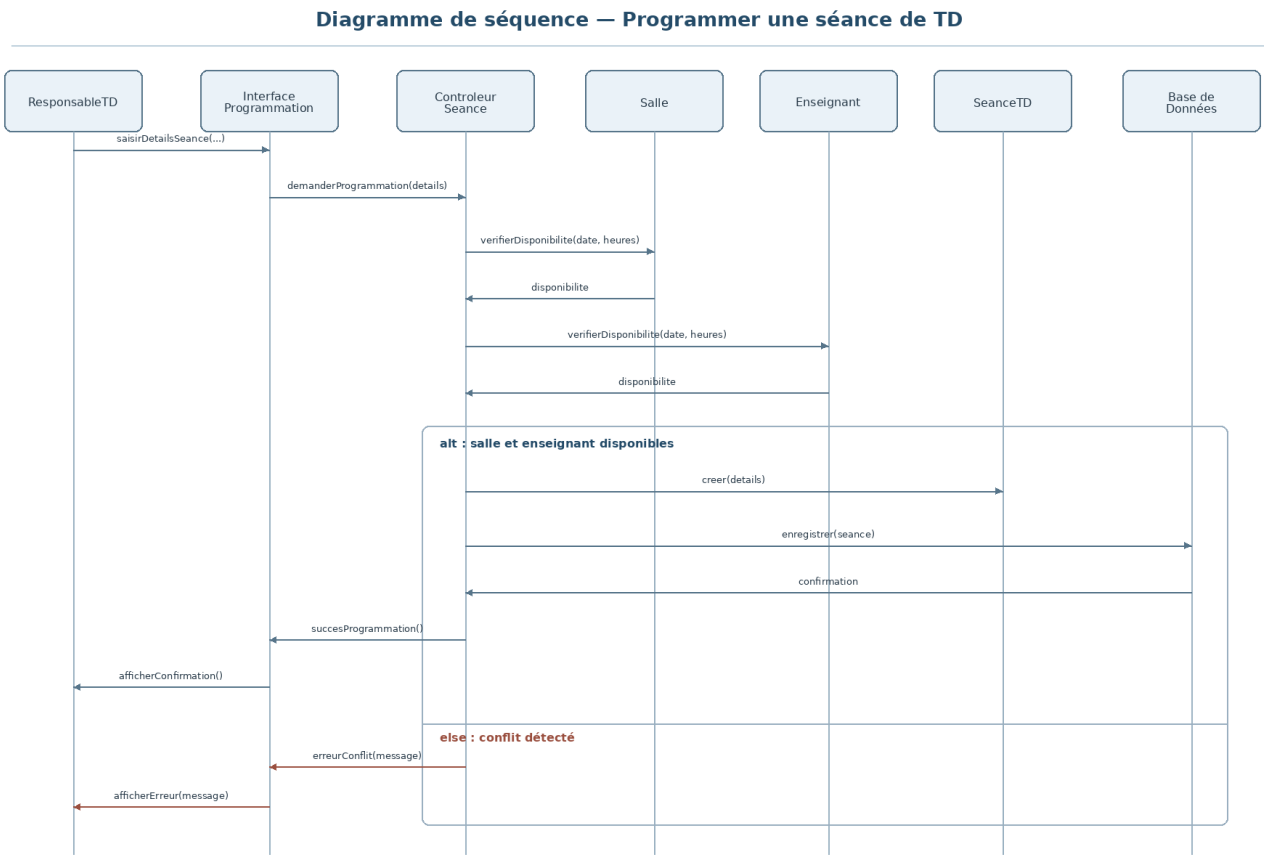


Diagramme 3 — Échanges lors de la programmation

6. Diagramme d’activités : déroulement d’un TD

L’enseignant ouvre la séance, pointe chaque étudiant, anime le TD et clôture la séance. Le système enregistre alors le bilan et change le statut en « terminée ». Le diagramme montre la boucle de pointage et la décision liée au temps imparti.

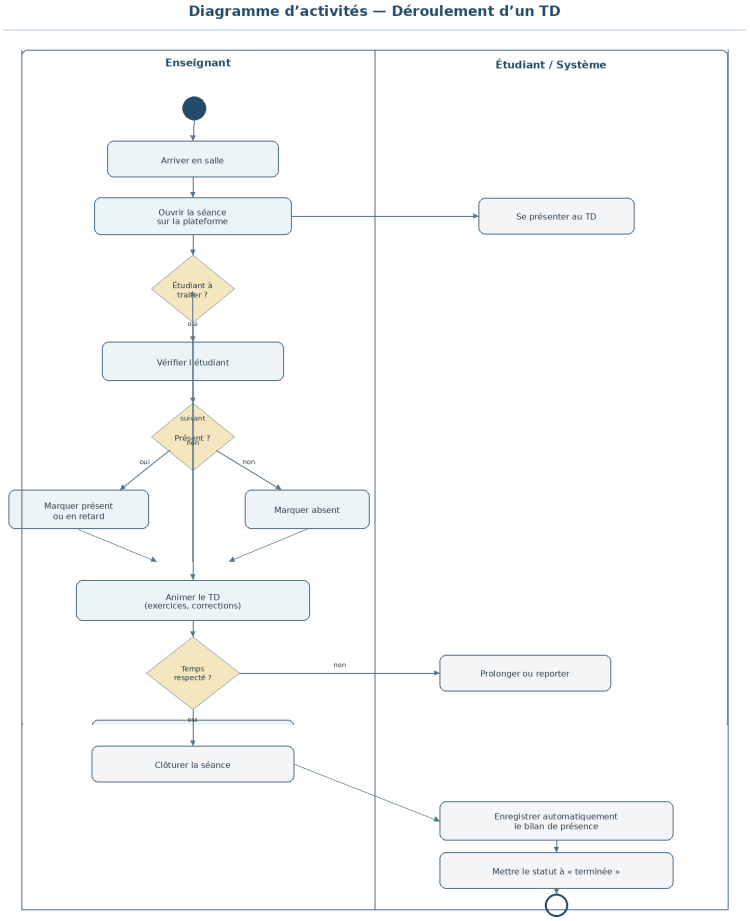


Diagramme 4 — Déroulement de la séance et couloirs d'activité

7. Extension : génération sans conflit

Deux intervalles d'une même date se chevauchent lorsque debutA < finB et debutB < finA. Le système refuse un créneau qui réserve une salle, un enseignant ou un groupe déjà occupé. La génération actuelle teste chaque salle pour la date et les heures demandées et retient la première disponible. Une évolution pourrait rechercher aussi les créneaux sur plusieurs jours et intégrer les capacités, disponibilités et préférences.

8. Réalisation et utilisation

Le projet est écrit en C++17. Le fichier unique src/main.cpp contient les déclarations des classes, l'implémentation métier et le menu console. Compiler avec make puis lancer ./gestion_td. Les dates sont au format AAAA-MM-JJ et les heures au format HH:MM.

9. Réponses aux questions du TP

Entités principales : Etudiant, Matiere, GroupeTD, SeanceTD, Enseignant, Salle et Presence.

Acteurs : Responsable TD, Enseignant et Étudiant. L'administrateur système est facultatif et n'est pas compris dans ce prototype.

Prévention des conflits : comparer date et horaires pour salle, enseignant et groupe; bloquer la séance en cas de chevauchement.

Génération automatique : tester les salles déclarées sur le créneau fourni et programmer la première salle libre, si le groupe et l'enseignant sont aussi disponibles.

Limites : absence de sauvegarde permanente, de comptes utilisateurs et de recherche automatique du créneau. Les évolutions recommandées sont une base de données, l'authentification et un planificateur de créneaux complet.

10. Conclusion

Les diagrammes décrivent les utilisateurs, les entités, le scénario de programmation et le déroulement d'un TD. Le prototype applique les principales règles de gestion et réalise une première recherche automatique de salle libre.